Автоматическая верификация оптимизаций LuaJIT с использованием SMT-решателя

А. С. Чуркин1, С. Бронников2, С. Каплун2
1 Московский физико-технический институт,
Россия, 141701, Долгопрудный, Институтский пер., д. 9.
2 ООО «VK»,
Россия, 191024, Санкт-Петербург, ул. Херсонская, д. 12-14.

Аннотация. Работа посвящена автоматической верификации оптимизаций JIT-компилятора LuaJIT с помощью SMT-решателя Z3. Ошибки в JIT-компиляторах приводят к трудно диагностируемым сбоям, поскольку некорректный машинный код генерируется только при определённых условиях, зависящих от входных данных и истории исполнения. Предложенный подход основан на проверке эквивалентности (translation validation): для каждой записанной трассы SSA IR строятся SMT-формулы, описывающие семантику до и после применения оптимизаций, и проверяется, что наблюдаемое поведение не изменилось. Реализован инструмент ljopt, кодирующий инструкции SSA IR LuaJIT на языке SMT-LIB в теориях с плавающей точкой, массивов, битовых векторов и алгебраических типов данных. Для корректного сопоставления формул учитывается специфика трасс LuaJIT: снимки состояния (snapshots), причины выхода с трассы и операции с таблицами. Эксперименты показывают, что инструмент способен автоматически обнаруживать известные ошибки в оптимизаторе LuaJIT и подтверждать корректность правильных оптимизаций на всех допустимых входах.

Ключевые слова: верификация компиляторов; JIT-компиляция; LuaJIT; SMT-решатель; проверка эквивалентности трансляции; символическое исполнение; SSA IR; Z3

Для цитирования: Чуркин А. С., Бронников С., Каплун С. Автоматическая верификация оптимизаций LuaJIT с использованием SMT-решателя. Труды ИСП РАН, том 37, вып. X, 2025 г., стр. xx–xx.

Automatic Verification of LuaJIT Optimizations Using an SMT Solver

A. S. Churkin1, S. Bronnikov2, S. Kaplun2
1 Moscow Institute of Physics and Technology,
9 Institutskiy per., Dolgoprudny, 141701, Russia.
2 VK LLC,
12-14 Khersonskaya st., Saint Petersburg, 191024, Russia.

Abstract. This paper presents an approach to automatic verification of JIT compiler optimizations in LuaJIT using the Z3 SMT solver. Bugs in JIT compilers lead to hard-to-diagnose failures because incorrect machine code is generated only under specific conditions that depend on input data and execution history. The proposed approach is based on translation validation: for each recorded SSA IR trace, SMT formulas describing the semantics before and after optimizations are constructed, and it is verified that the observable behavior has not changed. We implement the ljopt tool, which encodes LuaJIT SSA IR instructions into SMT-LIB using the theories of floating-point arithmetic, arrays, bit-vectors, and algebraic data types. To correctly match formulas, the tool accounts for LuaJIT-specific features: snapshots, trace exit conditions, and table operations. Experiments demonstrate that the tool can automatically detect known bugs in the LuaJIT optimizer and confirm the correctness of valid optimizations for all admissible inputs.

Keywords: compiler verification; JIT compilation; LuaJIT; SMT solver; translation validation; symbolic execution; SSA IR; Z3

For citation: Churkin A.S., Bronnikov S., Kaplun S. Automatic Verification of LuaJIT Optimizations Using an SMT Solver. Trudy ISP RAN/Proc. ISP RAS, vol. 37, issue X, 2025, pp. xx-xx (in Russian).

1. Введение

Компиляторы относятся к категории программного обеспечения, где критически важна корректность работы. Ошибки в компиляторах приводят к трудно диагностируемым сбоям в пользовательском коде и могут потребовать значительного времени для локализации. Особенно остро эта проблема проявляется в JIT-компиляторах, которые генерируют машинный код во время исполнения: ошибка может проявляться только при определённых профилях входных данных и истории исполнения.

LuaJIT — высокопроизводительный JIT-компилятор для языка Lua, широко применяемый в индустрии: в СУБД Tarantool, веб-сервере OpenResty, игровых движках и встраиваемых системах. JIT-компилятор LuaJIT использует трассирующую компиляцию: интерпретатор записывает горячие пути исполнения в виде линейных последовательностей инструкций SSA IR, к которым затем применяются оптимизации — свёртка констант, устранение избыточных проверок, алгебраические упрощения и другие.

Исследования в области повышения надёжности компиляторов ведутся с конца 2000-х годов. В 2009 году был создан CompCert [1] — первый формально верифицированный компилятор C. В 2021 году вышла работа Alive2 [2], посвящённая верификации оптимизаций LLVM IR. Аналогичные подходы реализованы для GCC [3] и PyPy [4]. Однако применительно к LuaJIT подобных инструментов до настоящего времени не существовало.

В данной работе представлен инструмент ljopt, реализующий автоматическую верификацию оптимизаций LuaJIT методом проверки эквивалентности трансляции (translation validation) с использованием SMT-решателя Z3 [5].

2. Подход

Предложенный подход основан на проверке эквивалентности: для одной и той же входной программы на Lua записываются трассы SSA IR с разными уровнями оптимизаций — без оптимизаций и с полным набором оптимизаций. Для каждой трассы строится SMT-формула, описывающая её семантику. Затем формулы объединяются в запрос к SMT-решателю: «существует ли вход, на котором наблюдаемое поведение различается?» Если решатель возвращает UNSAT — оптимизация корректна на всех допустимых входах. Если SAT — найден конкретный контрпример.

2.1 Кодирование инструкций SSA IR

Каждая инструкция SSA IR LuaJIT кодируется как набор SMT-утверждений. Для хранения значений операндов используется массив MemCell — алгебраический тип данных с конструкторами int-val (64-битный вектор), fp-val (число с плавающей точкой двойной точности) и str-val (строка). Арифметические операции над числами с плавающей точкой (ADD, SUB, MUL, DIV) кодируются соответствующими операциями теории FP в SMT-LIB: fp.add, fp.sub, fp.mul, fp.div с режимом округления RNE. Целочисленные операции кодируются в теории битовых векторов.

2.2 Моделирование таблиц

Таблицы Lua моделируются как трёхмерные массивы в теории массивов SMT-LIB: первый индекс — указатель на таблицу, второй — версия (для поддержки мутабельности), третий — ключ (MemCell). При каждой записи в таблицу версия инкрементируется, а все последующие чтения происходят из новой версии. Для корректной обработки ключей реализована нормализация: LuaJIT рассматривает -0.0 и +0.0 как один и тот же ключ, что учитывается при кодировании.

2.3 Снимки состояния и выходы с трассы

Ключевой особенностью трассирующего JIT-компилятора является наличие снимков состояния (snapshots) — точек, в которых трасса может быть прервана с возвратом в интерпретатор. Каждый снимок фиксирует значения переменных на стеке виртуальной машины. Для проверки эквивалентности необходимо сопоставить снимки двух трасс: обе трассы должны выходить по одному и тому же снимку с одинаковыми значениями переменных. Охранные условия (guards) кодируются как булевы формулы, а вектор выходов представляется битовым вектором.

3. Реализация

Инструмент ljopt реализован на языке Lua и состоит из следующих компонентов: дампер SSA IR, транслятор IR-to-SMT и модуль сравнения трасс. Дампер использует встроенный отладочный API LuaJIT для записи трасс. Транслятор обходит инструкции каждой трассы и генерирует SMT-утверждения.

Для повышения производительности SMT-решателя значения с плавающей точкой хранятся непосредственно как значения типа (_ FloatingPoint 11 53), без промежуточного преобразования в битовые векторы. Это позволяет Z3 использовать встроенные процедуры решения теории FP, что сокращает время решения на типичных трассах в десятки раз.

В текущей версии реализована поддержка более 30 инструкций SSA IR LuaJIT, включая арифметические операции, сравнения, преобразования типов, операции с таблицами (TNEW, TDUP, HREF, HREFK, HLOAD, HSTORE, AREF, ALOAD, ASTORE, FLOAD, FSTORE, NEWREF), операции со строками (TOSTR, STRTO, BUFHDR, BUFPUT, BUFSTR) и операции с FFI (CNEW, CNEWI).

4. Эксперименты

Для оценки инструмента были использованы известные ошибки в оптимизаторе LuaJIT, исправленные в разных версиях. Методика эксперимента: для каждой ошибки берётся версия LuaJIT до и после исправления, записываются трассы для воспроизводящей программы, строятся SMT-формулы и проверяется результат. На версии с ошибкой ожидается SAT (найден контрпример), на исправленной версии — UNSAT (оптимизация корректна).

Инструмент успешно обнаруживает ошибки в следующих категориях оптимизаций: свёртка констант с переполнением (ADDOV, SUBOV, MULOV), некорректное устранение проверок границ массивов (ABC), ошибки в правилах сужения типов (narrow), некорректные преобразования типов (CONV). Набор регрессионных тестов включает более 20 воспроизведённых ошибок.

Время решения одной SMT-формулы составляет от 0.1 до 30 секунд в зависимости от сложности трассы. Использование нативного типа с плавающей точкой вместо битовых векторов сокращает время решения в 10–40 раз на трассах с арифметическими операциями.

5. Заключение

В работе представлен инструмент ljopt для автоматической верификации оптимизаций JIT-компилятора LuaJIT. Инструмент кодирует инструкции SSA IR на языке SMT-LIB и проверяет эквивалентность трасс с помощью SMT-решателя Z3. Показано, что подход позволяет обнаруживать известные ошибки в оптимизаторе LuaJIT и подтверждать корректность оптимизаций на всех допустимых входах. Направления дальнейшей работы включают расширение покрытия инструкций IR, интеграцию с фаззером LuaJIT для автоматической генерации тестовых программ и применение инструмента для непрерывной верификации новых коммитов.

Список литературы / References

1. Leroy X. Formal verification of a realistic compiler. Communications of the ACM, vol. 52, no. 7, 2009, pp. 107-115.
2. Lopes N. P., Lee J., Hur C.-K., Liu Z., Regehr J. Alive2: Bounded Translation Validation for LLVM. In Proc. of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2021, pp. 65-79.
3. Walfridsson K. GCC Translation Validation. 2022. Доступно по ссылке: https://kristerw.github.io/2022/09/13/translation-validation/, дата обращения: 10.04.2025.
4. Bolz-Tereick C. F. Finding JIT Optimizer Bugs using SMT Solvers and Fuzzing. 2022. Доступно по ссылке: https://www.pypy.org/posts/2022/12/jit-bug-finding-smt-fuzzing.html, дата обращения: 10.04.2025.
5. de Moura L., Bjorner N. Z3: An Efficient SMT Solver. In Proc. of TACAS 2008, LNCS, vol. 4963, 2008, pp. 337-340.
6. d'Andrea D., Deniau L. Technical Documentation: Trace-Based Just-In-Time Compiler LuaJIT. 2020.

Информация об авторах / Information about authors

Алексей Сергеевич ЧУРКИН — студент магистратуры Московского физико-технического института. Сфера научных интересов: верификация компиляторов, JIT-компиляция, SMT-решатели.

Aleksey Sergeevich CHURKIN — graduate student at the Moscow Institute of Physics and Technology. Research interests: compiler verification, JIT compilation, SMT solvers.

Сергей БРОННИКОВ — инженер компании VK. Сфера научных интересов: тестирование компиляторов, фаззинг, LuaJIT.

Sergey BRONNIKOV — engineer at VK LLC. Research interests: compiler testing, fuzzing, LuaJIT.
Сергей КАПЛУН — инженер компании VK. Сфера научных интересов: JIT-компиляция, LuaJIT, оптимизация программ.
Sergey KAPLUN — engineer at VK LLC. Research interests: JIT compilation, LuaJIT, program optimization.